CENTRO UNIVERSITÁRIO DE VIÇOSA – UNIVIÇOSA
CURSO DE ENGENHARIA DA COMPUTAÇÃO

PEDRO ARTHUR OLIVEIRA JANUÁRIO

Mudanças no papel do desenvolvedor na era das IAs generativas: uma revisão da literatura sobre produtividade, prática profissional e impactos no desenvolvimento de software

Viçosa - MG
2026

PEDRO ARTHUR OLIVEIRA JANUÁRIO

Mudanças no papel do desenvolvedor na era das IAs generativas: uma revisão da literatura sobre produtividade, prática profissional e impactos no desenvolvimento de software

Trabalho de Conclusão de Curso apresentado ao Centro Universitário de Viçosa – Univiçosa como parte dos requisitos para a conclusão do curso de graduação em Engenharia da Computação.
Orientador: Anderson Resende Lamas

Viçosa - MG
2026

PEDRO ARTHUR OLIVEIRA JANUÁRIO

MUDANÇAS NO PAPEL DO DESENVOLVEDOR NA ERA DAS IAS GENERATIVAS: UMA REVISÃO DA LITERATURA SOBRE PRODUTIVIDADE, PRÁTICA PROFISSIONAL E IMPACTOS NO DESENVOLVIMENTO DE SOFTWARE

Trabalho de Conclusão de Curso apresentado ao Centro Universitário de Viçosa – Univiçosa como parte dos requisitos para a conclusão do curso de graduação em Engenharia da Computação.

Aprovado em: 03/09/2026

Banca examinadora:

Prof. Carlos Henrique Tavares Brumatti – Univiçosa

Prof. Me. Fabiano Mário Rodrigues Fialho – Univiçosa

Prof. Me. Anderson Resende Lamas
Orientador
Centro Universitário de Viçosa – Univiçosa

Resumo

Este trabalho apresenta uma revisão narrativa da literatura com apoio documental sobre as mudanças no papel do desenvolvedor na era da inteligência artificial (IA) generativa, com foco em produtividade, limitações técnicas, reorganização do processo de desenvolvimento e impactos na prática profissional. O núcleo da pesquisa reúne onze fontes, sendo cinco trabalhos acadêmicos e seis publicações técnicas, críticas e jornalísticas sobre assistentes de código, produtividade percebida e medida, riscos de retrabalho, *Spec-Driven Development* e *Harness Engineering*. A revisão indica que a IA generativa pode ampliar a produtividade em tarefas específicas, especialmente em contextos delimitados, repetitivos ou bem especificados. Entretanto, seus efeitos não são uniformes nem automáticos, pois dependem do tipo de tarefa, da complexidade do projeto, da qualidade das instruções e da experiência do desenvolvedor. Também se observa que o uso dessas ferramentas pode gerar retrabalho, exigir maior validação e criar uma falsa sensação de eficiência quando os resultados produzidos não são avaliados criticamente. Conclui-se que a IA não elimina a importância do profissional de software, mas desloca sua atuação para atividades mais estratégicas, como especificação, revisão, validação, arquitetura, supervisão de agentes e tomada de decisão técnica. Dessa forma, o uso eficiente da IA depende de conhecimento técnico, senso crítico e capacidade de estruturar adequadamente o trabalho antes, durante e depois da geração de código.

Palavras-chave: Inteligência artificial generativa; desenvolvimento de software; produtividade; Spec-Driven Development; papel do desenvolvedor.

Abstract

This work presents a narrative literature review with documentary support on changes in the developer's role in the era of generative artificial intelligence, focusing on productivity, technical limitations, process reorganization, and impacts on professional practice. The analytical core comprises eleven sources: five academic works and six technical, critical, and journalistic publications on coding assistants, perceived and measured productivity, rework risks, Spec-Driven Development, and Harness Engineering. The review indicates that generative AI can increase productivity in specific tasks, especially in delimited, repetitive, or well-specified contexts. However, its effects are not uniform or automatic, since they depend on the type of task, project complexity, quality of instructions, and developer experience. The literature also shows that these tools may generate rework, require stronger validation, and create a false sense of efficiency when their outputs are not critically evaluated. The study concludes that AI does not eliminate the relevance of software professionals, but shifts their work toward more strategic activities, such as specification, review, validation, architecture, agent supervision, and technical decision-making. Therefore, the effective use of AI depends on technical knowledge, critical judgment, and the ability to properly structure work before, during, and after code generation.

Keywords: Generative artificial intelligence; software development; productivity; Spec-Driven Development; developer role.

Lista de figuras

Figura 1 – Lançamentos oficiais ao público de ferramentas de desenvolvimento com IA. 14
Figura 2 – Ciclo simplificado de desenvolvimento orientado por especificação. . . . 19
Figura 3 – Níveis observados de *Spec-Driven Development*. 20
Figura 4 – Visão geral de *Harness Engineering* para usuários de agentes de código. 22

Sumário

1	INTRODUÇÃO	7
2	REVISÃO DA LITERATURA	9
2.1	Fontes utilizadas na revisão	9
2.2	A incorporação da IA generativa no desenvolvimento de software	11
2.2.1	Da sugestão de código à delegação de tarefas	13
2.3	IA generativa e produtividade no desenvolvimento de software	15
2.4	Limitações, riscos e ambiguidades do uso de IA na programação	16
2.5	*Spec-Driven Development* e a reorganização do desenvolvimento orientado por IA	18
2.6	*Harness Engineering* e controle de agentes de código	21
2.7	Transformações no papel do desenvolvedor na era da IA generativa	22
2.8	Síntese da revisão	24
3	METODOLOGIA	25
3.1	Caracterização da pesquisa	25
3.2	Constituição e delimitação das fontes	25
3.3	Procedimento de leitura e análise	26
3.4	Limitações do procedimento	27
4	ANÁLISE E DISCUSSÃO	28
4.1	Síntese dos principais achados	28
4.2	O paradoxo da produtividade: velocidade versus produtividade real	28
4.3	Retrabalho, validação e qualidade do software	29
4.4	A transformação do papel do desenvolvedor	30
4.5	Experiência profissional e dependência da IA	31
4.6	Spec-Driven Development e Harness Engineering como resposta à maior autonomia da IA	31
4.7	Implicações para formação e mercado de trabalho	32
4.8	Limitações da literatura analisada	33
5	CONCLUSÕES	34
5.1	Trabalhos futuros	36
	REFERÊNCIAS BIBLIOGRÁFICAS	37

1 Introdução

Nos últimos anos, a inteligência artificial (IA) generativa passou a ocupar um espaço cada vez mais relevante no desenvolvimento de software. Ferramentas capazes de gerar, revisar, explicar e refatorar código deixaram de ser vistas apenas como experimentos e passaram a fazer parte do cotidiano de estudantes, desenvolvedores e empresas. Com isso, o uso de IA na programação se tornou um dos temas mais discutidos atualmente, especialmente quando se consideram seus possíveis efeitos sobre a produtividade, a qualidade do desenvolvimento e a forma como o trabalho técnico vem sendo reorganizado.

Esse interesse não surgiu por acaso. À medida que essas ferramentas foram se tornando mais acessíveis e mais presentes no fluxo de trabalho, cresceu também a percepção de que elas podem acelerar diferentes tarefas da programação. O experimento de Peng et al. [2023] oferece uma evidência específica: participantes com acesso ao GitHub Copilot concluíram mais rapidamente uma tarefa controlada de programação do que aqueles sem esse suporte. Resultados como esse ajudam a explicar por que a IA passou a ser vista, por muitos profissionais e empresas, como um recurso com potencial para transformar o desenvolvimento de software.

Ainda assim, a discussão não pode ser reduzida à ideia de que gerar código mais rápido significa, necessariamente, desenvolver melhor. Os estudos de Stray et al. [2026] e Chen et al. [2026] mostram que compreender tarefas, colaborar, avaliar soluções e manter domínio sobre o trabalho são dimensões que não se resumem ao volume de código produzido. As discussões de Piskala [2026] e Böckeler [2026] acrescentam a importância de especificação, orientação e verificação nos fluxos com agentes. Por isso, mais do que perguntar se a IA consegue programar, torna-se necessário compreender como ela interfere no trabalho do desenvolvedor e quais mudanças ela produz na dinâmica de construção de software. Nesse ponto, a transformação em curso deixa de ser apenas uma questão de automação e passa a envolver também a forma de estruturar o próprio desenvolvimento.

Essa reorganização aparece com mais clareza em abordagens recentes que deslocam parte do foco da implementação direta para a especificação do problema, do contexto e dos critérios de validação. Entre essas abordagens, o *Spec-Driven Development* (SDD) vem sendo apresentado como uma prática em que a especificação passa a funcionar como referência central para orientar a geração, o planejamento e a verificação do código [PISKALA, 2026; DELIMARSKY, 2025]. Em vez de tratar a IA apenas como ferramenta para produzir trechos de código sob demanda, essa lógica destaca a importância de definir escopo, restrições, critérios de aceitação e contexto técnico antes da implementação. Isso reforça a hipótese de que a principal mudança trazida pela IA não está apenas em acelerar tarefas, mas em deslocar o valor do trabalho do desenvolvedor para atividades de

especificação, supervisão, validação e decisão.

A importância desse tema também se relaciona ao fato de que a adoção dessas ferramentas ocorre em ritmo acelerado e já influencia expectativas de mercado, práticas profissionais e formas de formação na área de tecnologia. No contexto recente do mercado brasileiro, esse avanço já aparece associado à pressão por maior produtividade, à redução do tamanho das equipes e à redefinição do perfil esperado dos desenvolvedores [GOMES; CORTIZ, 2026]. Ao mesmo tempo em que cresce o entusiasmo em torno da IA, também se ampliam questionamentos sobre seus limites, seus riscos e seus efeitos reais no processo de desenvolvimento. Em muitos casos, o debate público tende a simplificar a questão, tratando a IA apenas como sinônimo de velocidade ou, em sentido oposto, como ameaça imediata à profissão. Diante disso, torna-se importante analisar o fenômeno de forma mais equilibrada, evitando interpretações excessivamente otimistas ou reducionistas.

Nesse contexto, este trabalho tem como objetivo analisar como a IA generativa está impactando o desenvolvimento de software, especialmente em relação à produtividade, às limitações técnicas e à transformação do papel do desenvolvedor. Para isso, a pesquisa adota uma revisão narrativa da literatura com apoio documental, reunindo contribuições recentes da literatura acadêmica e fontes técnicas de apoio para construir uma análise crítica e atual sobre o tema. Parte-se da hipótese de que a IA pode ampliar a produtividade em diversas tarefas, mas não substitui o desenvolvedor. Em vez disso, sua presença tende a deslocar o foco da atuação profissional para funções mais estratégicas, como supervisão, validação, revisão e tomada de decisão técnica.

A partir dessa perspectiva, o problema de pesquisa que orienta este estudo é o seguinte: como a adoção de IA generativa está transformando o desenvolvimento de software e o papel do desenvolvedor, considerando ganhos de produtividade, limitações e mudanças na prática profissional? Para responder a essa questão, o trabalho está organizado em cinco capítulos. Após esta introdução, o Capítulo 2 apresenta as fontes e a revisão da literatura. O Capítulo 3 explicita a metodologia e seus limites. O Capítulo 4 compara e discute os achados, e o Capítulo 5 reúne as conclusões e as propostas de trabalhos futuros.

2 Revisão da Literatura

2.1 Fontes utilizadas na revisão

A análise apresentada neste trabalho articula onze fontes: cinco trabalhos acadêmicos, entre estudos empíricos e uma contribuição conceitual sobre *Spec-Driven Development*, e seis fontes técnicas, críticas e jornalísticas. O Quadro 1 identifica esse conjunto, em ordem crescente de ano dentro de cada grupo, e explicita a função de cada referência. Os estudos empíricos sustentam a discussão sobre produtividade e suas formas de avaliação; as contribuições conceituais e técnicas apoiam a análise da reorganização do desenvolvimento; e a fonte jornalística oferece contextualização do mercado brasileiro.

Essa distinção orienta a interpretação das evidências. Resultados de experimentos e estudos de caso são considerados em seus respectivos contextos, enquanto propostas técnicas, posicionamentos de profissionais e relatos jornalísticos são utilizados como apoio conceitual ou contextual. A inclusão de *preprints* no conjunto acadêmico não pressupõe que todos tenham passado pelo mesmo processo de avaliação por pares. O quadro apresenta as fontes utilizadas, sem caracterizar o levantamento como uma revisão sistemática ou como um inventário exaustivo da literatura.

Quadro 1 – Fontes que fundamentam a análise da revisão.

Autor/ano	**Título**	**Natureza**	**Contribuição para o trabalho**
Trabalhos acadêmicos			
Peng et al. [2023]	The Impact of AI on Developer Productivity: Evidence from GitHub Copilot	Experimento controlado; preprint.	Evidência de aceleração em uma tarefa delimitada de programação com Copilot.
Becker et al. [2025]	Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity	Experimento em projetos maduros; preprint.	Evidência de aumento do tempo de execução e limites da generalização dos ganhos.
Stray et al. [2026]	Developer Productivity With and Without GitHub Copilot: A Longitudinal Mixed-Methods Case Study	Estudo longitudinal; HICSS-59.	Contraste entre métricas de atividade baseadas em commits e percepção de produtividade.
Chen et al. [2026]	Beyond the Commit: Developer Perspectives on Productivity with AI Coding Assistants	Estudo de métodos mistos; versão em preprint.	Dimensões humanas da produtividade e limites das métricas exclusivamente baseadas em atividade.

Continua na próxima página.

Quadro 1 – continuação

Autor/ano	**Título**	**Natureza**	**Contribuição para o trabalho**
Piskala [2026]	Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants	Contribuição conceitual; preprint.	Base para discutir a especificação como referência de implementação e verificação.
Fontes técnicas, críticas e jornalísticas			
Delimarsky [2025]	Spec-driven development with AI: Get started with a new open source toolkit	Publicação oficial; GitHub Blog.	Exemplo de organização do desenvolvimento por especificações com o Spec Kit.
Böckeler [2025]	Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl	Análise técnica.	Diferentes interpretações de SDD e relação entre especificação, processo e ferramentas.
Thoughtworks [2025]	Spec-driven development	Avaliação técnica; Technology Radar.	Contextualização de SDD como abordagem emergente, com limites e diferentes aplicações.
Fowler [2026]	Fragments: January 8	Comentário crítico.	Contraponto à especificação inteiramente antecipada; importância de aprendizagem e feedback.
Gomes e Cortiz [2026]	IA revive medo da demissão: como dev é afetado por tecnologia feita por ele	Coluna jornalística; UOL Tilt.	Contexto brasileiro sobre expectativas de produtividade e transformação profissional.
Böckeler [2026]	Harness engineering for coding agent users	Artigo técnico conceitual.	Guias, sensores e feedback para orientar e verificar o trabalho de agentes de código.

Fonte: elaborado pelo autor a partir das referências indicadas no quadro.

Fontes documentais complementares

A contextualização histórica utiliza onze documentos oficiais sobre GitHub Copilot, Cursor, Trae, Augment Code, Claude Code, Codex, Kiro, OpenCode e Antigravity. Anúncios e retrospectivas fundamentam os lançamentos ao público, priorizando a disponibilidade geral; históricos de alterações e documentação apoiam a descrição funcional das ferramentas. A Subseção 2.2.1 identifica a natureza de cada evento e distingue datas de lançamento de atualizações posteriores.

Dois documentos adicionais, sobre Cobalt e Flightdeck, sustentam a discussão de ambientes e recursos apresentados em julho de 2026, na Seção 5.1. Sua função é indicar objetos de investigação futura, sem atribuir aos novos ambientes os resultados dos estu-

dos empíricos analisados.

Os documentos oficiais descrevem os produtos sob a perspectiva de seus responsáveis e não constituem evidência independente de ganho de produtividade. O núcleo analítico permanece composto pelas onze fontes do Quadro 1; os treze documentos complementares atendem à contextualização histórica e às perspectivas futuras. O Capítulo 3 apresenta a delimitação e o procedimento de análise desse material.

2.2 A incorporação da IA generativa no desenvolvimento de software

Nos últimos anos, a IA generativa passou de objeto experimental para elemento efetivo do desenvolvimento de software. Esse processo de popularização não ocorreu apenas por avanços técnicos nos modelos de linguagem, mas principalmente pela incorporação dessas tecnologias em ferramentas de uso cotidiano, capazes de sugerir código, explicar trechos, apoiar correções, gerar testes e acelerar tarefas recorrentes. Nesse contexto, o desenvolvimento de software deixou de lidar com a IA apenas como tema emergente e passou a incorporá-la como parte concreta do fluxo de trabalho profissional. Mais do que introduzir uma nova classe de ferramentas, essa mudança alterou a forma como várias etapas do desenvolvimento passaram a ser executadas, organizadas e avaliadas.

Parte da força desse movimento está relacionada à percepção de que a IA consegue reduzir o tempo necessário para a realização de determinadas atividades de programação. Peng et al. [2023], ao investigarem o uso do GitHub Copilot em uma tarefa controlada, observaram que participantes com acesso à ferramenta concluíram a atividade mais rapidamente do que aqueles sem acesso ao sistema. Embora o estudo esteja situado em um contexto específico, seus resultados ajudam a compreender por que a IA generativa ganhou tanta relevância no desenvolvimento de software em um intervalo relativamente curto. A possibilidade de acelerar tarefas pontuais, especialmente aquelas mais repetitivas ou mais estruturadas, contribuiu para consolidar a ideia de que essas ferramentas poderiam alterar de maneira concreta a produtividade e a dinâmica do trabalho técnico.

Essa aceleração, porém, não deve ser compreendida apenas como aumento de velocidade operacional. A popularização dessas ferramentas também reconfigurou o fluxo de trabalho de desenvolvedores e equipes. Em vez de depender exclusivamente de escrita manual, busca por documentação, consulta a fóruns e reaproveitamento artesanal de soluções anteriores, o desenvolvedor passou a contar com sistemas capazes de responder em linguagem natural, propor estruturas iniciais e antecipar parte do trabalho que antes exigia maior esforço mecânico. Isso modificou não apenas o tempo gasto em certas tarefas, mas também a distribuição da atenção ao longo do processo de desenvolvimento. Em muitos casos, a atividade passou a envolver menos digitação direta e mais seleção, adaptação,

revisão e validação do que foi produzido com apoio da ferramenta. Dessa forma, a IA generativa começou a interferir não apenas no produto do trabalho, mas na própria lógica de execução do trabalho técnico.

Os reflexos dessa mudança apareceram rapidamente também no mercado. Gomes e Cortiz [2026], ao discutirem o impacto recente da IA sobre a carreira de desenvolvedores, apresentam um cenário em que a pressão por produtividade aumenta ao mesmo tempo em que empresas passam a operar com equipes menores e com maior cobrança por entrega. A matéria mostra que a difusão da IA foi associada a uma nova régua de desempenho, em que tarefas antes planejadas para períodos mais longos passaram a ser vistas como passíveis de execução em ciclos mais curtos. Além disso, a discussão apresentada pelos autores reforça que o impacto da IA não se limita à automação pontual de tarefas, mas alcança decisões organizacionais sobre contratação, composição de times e perfil profissional esperado. Nesse sentido, a popularização da IA generativa contribuiu para deslocar o debate do campo puramente técnico para um campo também econômico e profissional.

Esse processo ajuda a explicar por que a adoção dessas tecnologias passou a ser percebida não apenas como ganho instrumental, mas como transformação do ambiente de desenvolvimento. A incorporação de assistentes baseados em IA passou a criar expectativa de maior autonomia para a execução de tarefas, sobretudo na produção de estruturas iniciais, rotinas repetitivas e apoio à resolução de problemas localizados. Ao mesmo tempo, a expansão dessas ferramentas também trouxe uma mudança mais ampla na forma de organizar o trabalho, já que parte do valor do desenvolvimento começou a migrar da escrita manual de código para a capacidade de orientar melhor a ferramenta, definir contexto e supervisionar resultados. É nesse ponto que contribuições recentes sobre fluxos orientados por especificação se tornam úteis como apoio conceitual, ao destacar que ambientes com agentes de IA tendem a valorizar mais definição de intenção, explicitação de contexto e estruturação prévia do trabalho [DELIMARSKY, 2025; BÖCKELER, 2025].

Mesmo quando utilizada como apoio contextual, essa leitura é importante porque mostra que a transformação em curso não se resume ao surgimento de ferramentas mais rápidas, mas envolve uma reorganização do próprio processo de desenvolvimento. A popularização de soluções como GitHub Copilot, Codex, Cursor e Claude Code indica que a IA generativa passou a atuar como componente real do ambiente de programação, interferindo em práticas, expectativas e formas de produzir software. Assim, compreender como essas ferramentas se difundiram e como passaram a alterar o fluxo de trabalho constitui um passo inicial necessário para a revisão da literatura. A partir desse ponto, torna-se possível avançar para uma questão mais específica e central no debate recente: em que medida essa incorporação da IA generativa efetivamente se traduz em aumento de produtividade no desenvolvimento de software.

2.2.1 Da sugestão de código à delegação de tarefas

A incorporação da IA ao desenvolvimento reúne formas de trabalho que coexistem: sugestões durante a escrita, edição orientada por instruções e delegação de tarefas a agentes. A Figura 1 organiza os lançamentos oficiais ao público, priorizando a disponibilidade geral quando anunciada pelos responsáveis. A sequência temporal não representa substituição obrigatória entre produtos nem demonstra que ferramentas posteriores sejam necessariamente mais produtivas.

Em 21 de junho de 2022, o GitHub Copilot passou à disponibilidade geral para desenvolvedores individuais, com sugestões de código no editor [DOHMKE, 2022]. Em 24 de outubro de 2024, o lançamento comercial do Augment Code apresentou extensões para editores, sugestões contextualizadas e planos para equipes, com inscrição para avaliação do produto [KELLY, 2024]. A retrospectiva oficial do Trae registra seu lançamento em janeiro de 2025, sem informar o dia [The TRAE Team, 2025].

O Claude Code chegou à disponibilidade geral em 22 de maio de 2025, ampliando a colaboração com agentes no terminal e em outros fluxos de desenvolvimento [Anthropic, 2025]. Em 19 de junho de 2025, Dax Raad anunciou que a versão reescrita do OpenCode estava pronta para uso geral, com interface de terminal e suporte a diferentes provedores de modelos [RAAD, 2025]. Esse marco corresponde à liberação dessa versão ao público, e não ao início do desenvolvimento do projeto. Em 6 de outubro de 2025, a OpenAI anunciou a disponibilidade geral do Codex, com atuação no editor, no terminal e na nuvem [OpenAI, 2025]. Neste recorte, Codex designa o agente de programação e suas interfaces de uso; não o modelo homônimo de geração de código original.

O Kiro chegou à disponibilidade geral em 17 de novembro de 2025, reunindo desenvolvimento orientado por especificações, testes de propriedades e agentes no ambiente de desenvolvimento e no terminal [SWAMINATHAN; ILINCA, 2025]. Em 18 de novembro de 2025, o Antigravity foi lançado ao público, articulando edição e coordenação de agentes com atuação no editor, no terminal e no navegador. Nesse caso, o anúncio oficial caracteriza o lançamento como prévia pública, e não como disponibilidade geral [The Antigravity Team, 2025]. Esses marcos ampliam o conjunto de atividades que pode ser delegado, incluindo etapas de execução e verificação além da geração inicial.

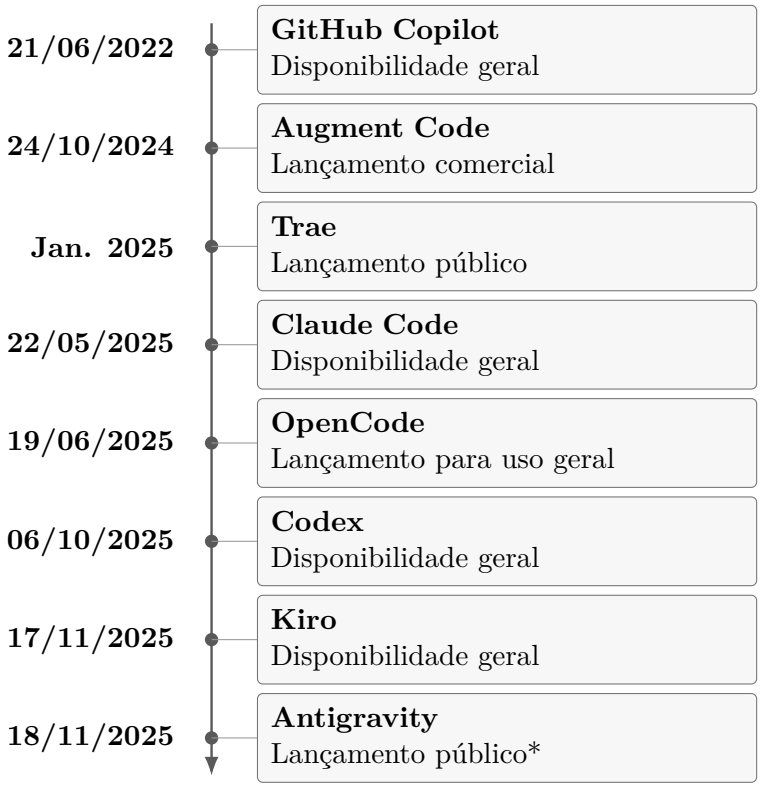


Figura 1 – Lançamentos oficiais ao público de ferramentas de desenvolvimento com IA.

Fonte: elaborado pelo autor com base em Dohmke [2022], Kelly [2024], The TRAE Team [2025], Anthropic [2025], Raad [2025], OpenAI [2025], Swaminathan e Ilinca [2025] e The Antigravity Team [2025].

Nota: espaçamento sem escala temporal proporcional. *Antigravity: lançamento em prévia pública. OpenCode: liberação da versão reescrita para uso geral. Trae: mês e ano conforme a fonte. Cursor permanece no texto, sem data de estreia confirmada.

O histórico oficial do Cursor documenta edição por instruções em julho de 2023, mas esse registro é uma atualização de versão, não um anúncio de estreia [Cursor, 2023]. Por esse motivo, o Cursor permanece na contextualização textual. A documentação atual do OpenCode caracteriza um agente de código aberto com diferentes interfaces de uso [OpenCode, s.d.]; a do Augment Code descreve agentes e recursos de contexto sobre o projeto [Augment Code, s.d.]. Essas descrições atuais não implicam que todos os recursos já existissem nos lançamentos indicados.

O conjunto permite observar uma ampliação das possibilidades de interação: o desenvolvedor pode aceitar sugestões locais, solicitar alterações e acompanhar a execução de uma tarefa mais abrangente. Essas capacidades não dispensam definição de objetivos, revisão e validação. As descrições oficiais sustentam a caracterização dos produtos, mas os efeitos sobre produtividade precisam ser examinados pelas evidências empíricas discutidas a seguir.

2.3 IA generativa e produtividade no desenvolvimento de software

A relação entre IA generativa e produtividade tornou-se uma das questões centrais no debate recente sobre desenvolvimento de software. Parte importante desse interesse decorre de estudos que identificam ganhos concretos de desempenho em tarefas específicas com o uso de assistentes de código baseados em IA. Peng et al. [2023], ao investigarem o uso do GitHub Copilot em uma tarefa controlada de programação, observaram que participantes com acesso à ferramenta concluíram a atividade em menos tempo do que aqueles sem esse suporte. Esse resultado contribuiu para consolidar a percepção de que a IA generativa pode acelerar etapas do trabalho técnico, sobretudo em contextos estruturados e em tarefas com maior previsibilidade.

Entretanto, os próprios desdobramentos da literatura mostram que a discussão sobre produtividade não pode ser reduzida à ideia de aceleração imediata de tarefas. Em engenharia de software, produtividade envolve não apenas tempo de execução, mas também qualidade da solução, aderência ao contexto do projeto, facilidade de manutenção, coerência entre componentes e impacto do trabalho ao longo do ciclo de vida do sistema. Nesse sentido, embora ferramentas de IA possam reduzir esforço inicial em determinadas atividades, isso não significa automaticamente ganho líquido de produtividade em todos os cenários. A questão central, portanto, não é apenas se a IA acelera o desenvolvimento, mas em que condições essa aceleração representa produtividade real.

Essa complexidade aparece de forma mais nítida em estudos que distinguem produtividade percebida de produtividade medida. Stray et al. [2026], em um estudo longitudinal sobre uso do GitHub Copilot, mostram que a adoção da ferramenta não produziu mudanças estatisticamente significativas em métricas de atividade baseadas em *commits* após sua incorporação ao trabalho. O estudo é importante porque relativiza a narrativa segundo a qual o uso de IA sempre resulta em aumento objetivo de produtividade. Nesse caso, embora a ferramenta estivesse associada à percepção de apoio ao trabalho, isso não se traduziu de forma clara em indicadores mensuráveis de atividade. O resultado reforça a ideia de que produtividade em desenvolvimento de software é um conceito difícil de capturar e não pode ser inferido apenas pela adoção da tecnologia.

A distinção entre percepção e medição também aparece em Chen et al. [2026], que discutem a produtividade com assistentes de código por IA a partir da perspectiva dos próprios desenvolvedores. Os autores mostram que a experiência de produtividade relatada pelos usuários envolve diferentes dimensões, incluindo fluidez no início de tarefas, sensação de continuidade no trabalho, redução de esforço cognitivo em certas etapas e facilidade para explorar alternativas. Ao mesmo tempo, o estudo indica que essas percepções não podem ser resumidas a uma métrica simples, justamente porque o trabalho em software inclui atividades diversas, com exigências distintas e efeitos que nem sempre

aparecem de forma imediata em indicadores tradicionais. Assim, a produtividade com IA não deve ser tratada apenas como resultado quantitativo, mas como fenômeno mais amplo, marcado por variações conforme contexto, tarefa e perfil de uso.

Além disso, parte da literatura recente aponta que o impacto da IA pode ser negativo em determinados ambientes de desenvolvimento. Becker et al. [2025], ao analisarem desenvolvedores experientes em projetos de código aberto maduros, encontraram um cenário em que o uso de IA não acelerou o trabalho, mas aumentou o tempo necessário para a realização das tarefas. Esse aumento está relacionado ao fato de que, em projetos maiores, com padrões de qualidade consolidados e forte dependência de conhecimento contextual, o tempo economizado na geração inicial de código pode ser consumido pela necessidade de compreender, conferir e adaptar as sugestões da ferramenta ao ambiente real do projeto. Assim, o estudo afasta a ideia de que a produtividade proporcionada pela IA seja uniforme, automática ou independente do contexto em que ela é aplicada.

A partir desses estudos, observa-se que a IA generativa pode ampliar a produtividade em determinadas atividades, especialmente naquelas mais repetitivas, mais previsíveis ou mais delimitadas. No entanto, também se evidencia que esse ganho não deve ser generalizado sem considerar a natureza da tarefa, o tipo de projeto, a experiência do desenvolvedor e os critérios adotados para medir produtividade. Em vez de uma resposta definitiva, a literatura oferece um quadro mais nuançado: a IA pode reduzir esforço e tempo em alguns cenários, mas seus efeitos reais dependem de condições específicas e da forma como o trabalho técnico é organizado. Essa constatação leva, naturalmente, à necessidade de examinar com mais atenção os limites, os riscos e as ambiguidades associados ao uso dessas ferramentas.

2.4 Limitações, riscos e ambiguidades do uso de IA na programação

Se por um lado a literatura recente reconhece o potencial da IA generativa para acelerar determinadas tarefas, por outro ela também evidencia limitações importantes em seu uso no desenvolvimento de software. Uma das questões mais recorrentes diz respeito à confiabilidade das saídas produzidas. Ferramentas baseadas em modelos generativos podem oferecer respostas plausíveis, bem escritas e aparentemente completas, mas que não necessariamente são corretas, seguras ou adequadas ao contexto do sistema em desenvolvimento. Essa característica faz com que o valor imediato da resposta gerada precise ser sempre acompanhado de verificação técnica, especialmente em ambientes mais complexos.

Os riscos se tornam mais visíveis quando se considera que a aparência de completude da resposta pode induzir à aceitação acrítica do conteúdo produzido. Chen et al. [2026] mostram que a percepção de produtividade associada a assistentes de código envolve

elementos subjetivos relevantes, como sensação de fluidez e facilidade para avançar em tarefas. No entanto, justamente por isso, existe o risco de o desenvolvedor interpretar rapidez de resposta como sinônimo de qualidade ou adequação técnica. Quando isso ocorre, a IA pode produzir uma falsa sensação de eficiência: o trabalho parece avançar mais rapidamente, mas parte do esforço apenas é deslocada para etapas posteriores de correção, revisão e manutenção.

Essa ambiguidade se torna ainda mais clara em contextos de maior complexidade. Becker et al. [2025], ao analisarem o uso de IA por desenvolvedores experientes em projetos de código aberto, mostram que a ferramenta pode aumentar o tempo necessário para concluir tarefas em vez de reduzi-lo. Esse achado é importante porque indica que, em certos ambientes, a IA não simplifica o trabalho, mas acrescenta novas camadas de esforço, como conferência de aderência ao contexto do projeto, correção de sugestões inadequadas e validação de saídas que parecem corretas apenas superficialmente. Em vez de eliminar complexidade, a IA pode antecipar a produção visível de código ao mesmo tempo em que preserva ou até amplia a complexidade invisível do trabalho posterior.

Outro ponto relevante na literatura diz respeito à relação entre experiência prévia e uso efetivo dessas ferramentas. A facilidade com que a IA gera soluções iniciais pode ser especialmente atraente para usuários menos experientes, pois reduz barreiras de entrada e amplia a capacidade de produzir resultados rápidos. No entanto, essa mesma facilidade pode ocultar lacunas de compreensão, uma vez que obter uma resposta não é o mesmo que entender a lógica, os limites e as implicações do que foi produzido. Por outro lado, desenvolvedores mais experientes podem se beneficiar da aceleração em tarefas pontuais, mas também tendem a perceber com mais clareza os problemas de aderência, manutenção e consistência gerados por respostas inadequadas. Assim, o efeito da IA não depende apenas da ferramenta, mas também da capacidade do usuário de avaliar criticamente sua produção.

Stray et al. [2026] reforçam essa leitura ao mostrar que a adoção do Copilot não se traduziu automaticamente em mudanças estatisticamente significativas em métricas de atividade. Esse resultado sugere que o uso da IA não deve ser interpretado como solução imediata para aumento de desempenho em qualquer contexto. Ao contrário, seus efeitos são heterogêneos e podem variar de acordo com o tipo de trabalho, a forma de integração da ferramenta ao fluxo de desenvolvimento e os critérios utilizados para medir resultados. A literatura, portanto, não sustenta uma visão linear da IA como mecanismo universal de eficiência.

Desse modo, os estudos revisados indicam que o uso de IA generativa em programação é marcado por ambiguidades. Em determinadas tarefas, ela pode reduzir esforço e tempo; em outras, pode gerar retrabalho, elevar a necessidade de supervisão e dificultar a avaliação de qualidade. Essa ambivalência é central para o presente trabalho, porque mostra que o debate não deve ser formulado em termos binários, como se a IA apenas ajudasse

ou apenas prejudicasse. O que emerge da literatura é a necessidade de compreender os limites dessas ferramentas, os contextos em que elas efetivamente contribuem e os riscos associados à sua adoção sem mediação crítica.

2.5 *Spec-Driven Development* e a reorganização do desenvolvimento orientado por IA

No contexto do desenvolvimento assistido por IA, o *Spec-Driven Development* (SDD) emerge como uma abordagem que desloca o centro do trabalho da codificação imediata para a explicitação prévia do problema. Em vez de tratar o código como artefato primário e a especificação como documentação secundária, a proposta do SDD é inverter essa lógica: a especificação passa a funcionar como fonte de verdade para orientar planejamento, implementação e validação, enquanto o código é produzido e ajustado a partir desse referencial [PISKALA, 2026].

Esse deslocamento ganhou força com a expansão dos *coding agents*. Neste trabalho, o termo é utilizado para se referir a ferramentas baseadas em IA capazes de atuar com maior autonomia no processo de desenvolvimento, indo além da simples sugestão de trechos de código. Esses agentes podem interpretar instruções em linguagem natural, consultar arquivos do projeto, propor alterações, modificar código, executar comandos ou testes e iterar a partir de mensagens de erro ou *feedback* do usuário, sempre dependendo de supervisão humana. Em fluxos com agentes, quanto mais ambíguo for o pedido inicial, maior a chance de o modelo preencher lacunas com suposições incorretas. Por isso, abordagens orientadas por especificação vêm sendo apresentadas como estratégia para reduzir ruído de contexto e melhorar previsibilidade, ao estruturar requisitos, critérios de aceitação, restrições e decisões técnicas antes da execução das tarefas [DELIMARSKY, 2025; Thoughtworks, 2025].

A Figura 2 sintetiza esse deslocamento ao representar um ciclo em que a especificação antecede a implementação e passa a orientar as etapas seguintes. Nesse fluxo, os objetivos e o contexto do problema são explicitados antes da geração ou adaptação do código. Em seguida, o planejamento organiza tarefas e restrições, enquanto a validação por testes e revisão retroalimenta a própria especificação. A figura ilustra, portanto, que o SDD não elimina a implementação, mas reorganiza sua posição dentro do processo.

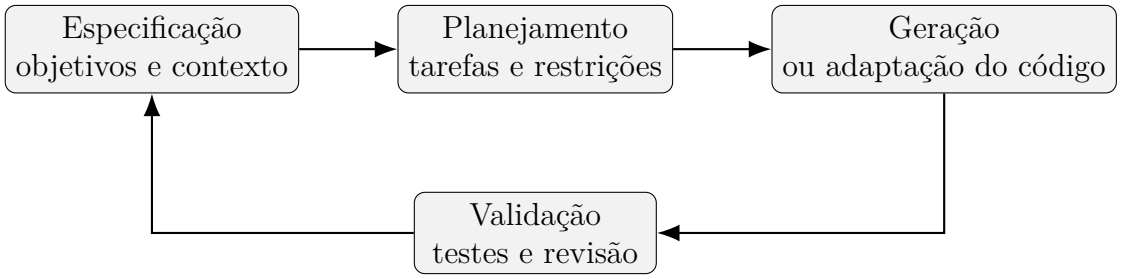


Figura 2 – Ciclo simplificado de desenvolvimento orientado por especificação. Fonte: elaborado pelo autor com base em Piskala [2026], Delimarsky [2025] e Böckeler [2025].

Nesse sentido, o SDD não deve ser confundido com a ideia de apenas escrever *prompts* mais longos. A literatura prática sobre o tema indica que o ponto central não é a verbosidade do comando, mas a organização do processo em artefatos que permitam rastreabilidade entre intenção, plano e implementação. Isso inclui decompor objetivos, tornar explícitas dependências técnicas, definir regras de qualidade e revisar iterativamente os resultados gerados pelos agentes [BÖCKELER, 2025]. Assim, o ganho potencial do SDD está menos em "pedir melhor" e mais em estabelecer um ciclo de desenvolvimento com menos ambiguidade estrutural.

A Figura 3 apresenta uma visão complementar desse processo ao diferenciar níveis de adoção do *Spec-Driven Development*. No nível *spec-first*, a especificação orienta a criação inicial da funcionalidade, mas pode ser descartada depois. No nível *spec-anchored*, a especificação permanece como referência durante a evolução e manutenção. Já no nível *spec-as-source*, a especificação passa a funcionar como fonte central para orientar a geração e a atualização do código. Essa distinção é relevante para o presente trabalho porque mostra que a especificação pode deixar de ser apenas documentação auxiliar e se tornar parte ativa do fluxo de desenvolvimento com IA.

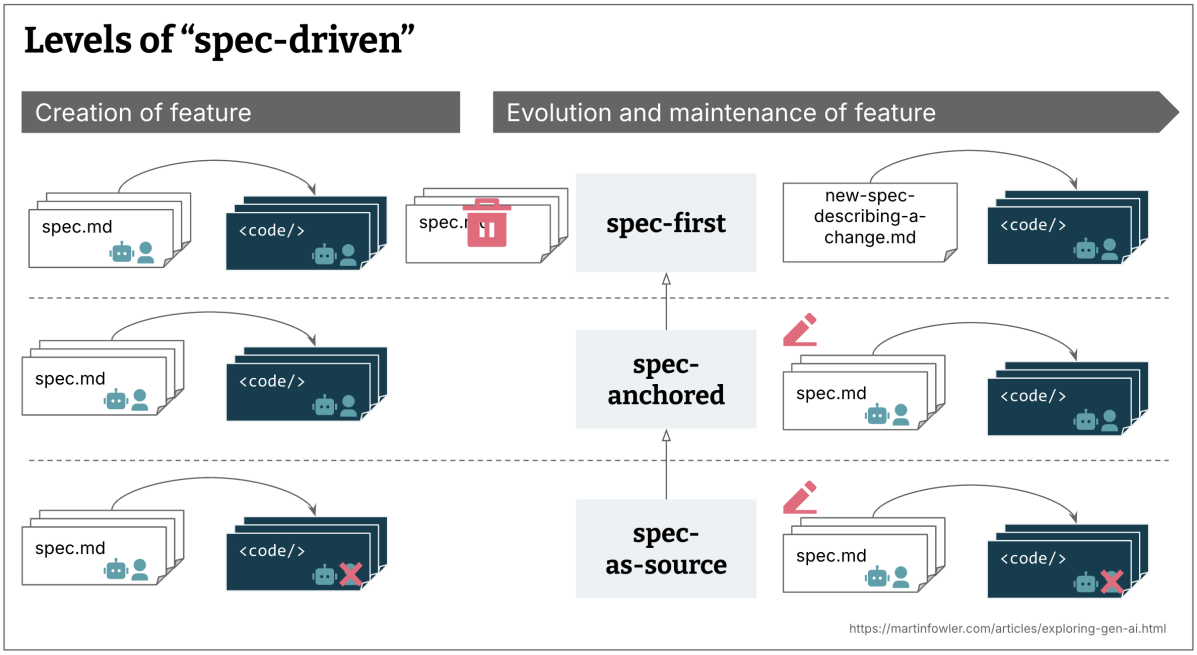


Figura 3 – Níveis observados de *Spec-Driven Development*. Fonte: Böckeler [2025].

Ao mesmo tempo, esse movimento não deve ser tratado como solução universal. Em contraponto, Fowler [2026] reproduz uma observação de Kent Beck de que uma visão excessivamente linear de "especificar tudo antes" pode ignorar o aprendizado que ocorre durante a implementação. Essa ressalva é relevante porque desenvolvimento de software envolve iteração, descoberta de restrições e revisão de hipóteses ao longo do caminho [FOWLER, 2026]. Portanto, uma leitura equilibrada do SDD exige entendê-lo como abordagem útil para estruturar trabalho com IA, mas não como substituto de *feedback* contínuo e experimentação.

A principal contribuição do SDD para este trabalho está em reforçar a hipótese de transformação do papel do desenvolvedor. Quando a especificação, o contexto e os critérios de validação se tornam centrais, o valor da atuação profissional se desloca da execução manual contínua para atividades de definição, coordenação, supervisão e decisão técnica. Dessa forma, o SDD ajuda a explicar por que a adoção de IA tende a reorganizar o processo de desenvolvimento e a aumentar a importância de competências analíticas e arquiteturais no cotidiano do desenvolvedor. Essa discussão conduz à necessidade de analisar também os mecanismos de controle ao redor dos agentes de código, tema tratado pela abordagem de *Harness Engineering*.

2.6 *Harness Engineering* e controle de agentes de código

O *Harness Engineering*, discutido no artigo de Böckeler [2026][1], surge como uma abordagem complementar ao SDD para lidar com o uso de agentes de código em ambientes reais de desenvolvimento. Enquanto o SDD enfatiza a importância de especificar objetivos, contexto e critérios antes da implementação, o *Harness Engineering* concentra-se na construção de estruturas de orientação e verificação ao redor dos agentes. A ideia central é que, quanto maior a autonomia concedida à IA, maior também deve ser a presença de mecanismos capazes de guiar sua atuação, detectar problemas e permitir correções durante o processo [BÖCKELER, 2026].

Nessa abordagem, o agente de código não é tratado como elemento isolado. Ele opera dentro de um conjunto de guias, sensores e ciclos de *feedback*. Os guias funcionam como mecanismos de orientação prévia, como regras de projeto, documentação, exemplos, arquivos de contexto, padrões arquiteturais e especificações. Já os sensores funcionam como mecanismos de verificação posterior, incluindo testes automatizados, *linters*, análise estática, *logs*, revisão humana e outros recursos capazes de identificar inconsistências ou falhas. Dessa forma, *Harness Engineering* reforça a ideia de que o uso produtivo da IA depende menos de confiança irrestrita na geração automática e mais da criação de um ambiente técnico capaz de controlar, avaliar e corrigir as saídas produzidas.

A Figura 4 ilustra essa organização ao apresentar uma visão geral de *Harness Engineering* para usuários de agentes de código. Na figura, os guias aparecem como elementos que alimentam previamente o agente, oferecendo contexto e restrições para a geração. Os sensores aparecem como mecanismos de *feedback*, responsáveis por avaliar o resultado e apoiar ciclos de autocorreção. A presença do usuário humano na estrutura reforça que o desenvolvedor continua responsável por orientar o processo, selecionar os mecanismos adequados e decidir quando a saída produzida pode ser aceita.

[1] Texto de apoio: <https://martinfowler.com/articles/harness-engineering.html>.

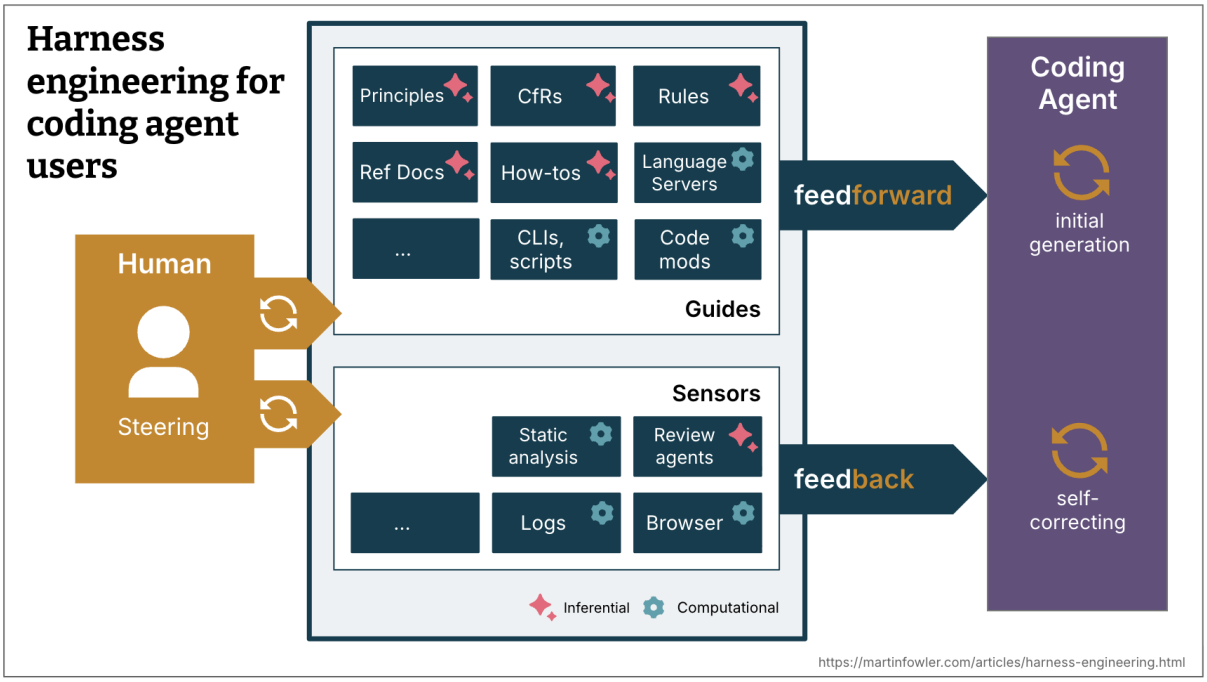


Figura 4 – Visão geral de *Harness Engineering* para usuários de agentes de código. Fonte: Böckeler [2026].

Essa abordagem é relevante para o presente trabalho porque aprofunda a hipótese de que a IA não elimina o papel do desenvolvedor, mas desloca sua atuação para funções de especificação, supervisão e validação. Em fluxos baseados em agentes, o desenvolvedor precisa definir o contexto de trabalho, escolher quais guias serão disponibilizados, interpretar os sinais produzidos pelos sensores e decidir se a solução gerada atende aos critérios técnicos do projeto. Assim, *Harness Engineering* ajuda a explicar por que a produtividade com IA não depende apenas da capacidade de gerar código rapidamente, mas também da qualidade dos mecanismos de controle que sustentam essa geração.

2.7 Transformações no papel do desenvolvedor na era da IA generativa

Além do debate sobre produtividade, a literatura e as fontes contextuais revisadas indicam que a incorporação da IA generativa vem alterando o próprio papel do desenvolvedor no processo de construção de software. Ao discutir o fluxo centrado no código, Piskala [2026] contrapõe a implementação como referência principal à proposta de orientar o desenvolvimento por especificações. Essa distinção ajuda a compreender o deslocamento de responsabilidades discutido neste trabalho, sem reduzir a atuação anterior do desenvolvedor à digitação de código. Com a presença crescente de assistentes baseados em IA, parte dessa atividade passa a ser mediada por ferramentas capazes de sugerir estruturas, antecipar soluções e responder a comandos em linguagem natural. Como consequência, o foco

do trabalho tende a se deslocar, ao menos parcialmente, da produção manual contínua para a orientação, avaliação e integração do que é produzido com apoio desses sistemas.

Esse deslocamento não implica desaparecimento do desenvolvedor, mas reconfiguração de sua função. Gomes e Cortiz [2026], ao discutir o impacto da IA sobre o mercado de tecnologia, mostram que o avanço dessas ferramentas vem sendo associado à redução do tamanho das equipes, à elevação da cobrança por produtividade e à redefinição do perfil profissional esperado. Nesse contexto, ganha força a percepção de que o valor do desenvolvedor não está apenas em escrever código, mas em saber conduzir o processo técnico de maneira mais qualificada. A pressão deixa de se concentrar exclusivamente na capacidade de implementar manualmente e passa a incluir também julgamento técnico, capacidade de supervisão e responsabilidade sobre a qualidade final do sistema.

Essa interpretação encontra apoio conceitual em discussões recentes sobre fluxos orientados por especificação e agentes de IA. Nessa lógica, a especificação deixa de ser apenas documento auxiliar e passa a funcionar como elemento central para coordenar o comportamento dos sistemas de IA [PISKALA, 2026; DELIMARSKY, 2025]. Isso reforça a ideia de que o papel do desenvolvedor tende a se tornar menos centrado em implementação mecânica e mais orientado à formulação de problemas, à estruturação do contexto e à validação das soluções produzidas.

A própria literatura sobre produtividade e limitações ajuda a sustentar esse deslocamento. Se os ganhos de desempenho com IA dependem da capacidade de revisar, corrigir e contextualizar as respostas geradas, então o conhecimento técnico do desenvolvedor não perde relevância. Ao contrário, ele se torna ainda mais necessário. Estudos como os de Chen et al. [2026] e Becker et al. [2025] indicam, de formas diferentes, que a utilidade da IA está diretamente ligada à capacidade do usuário de interpretar criticamente suas saídas. Isso significa que o desenvolvedor continua sendo peça central do processo, não como mero digitador de código, mas como agente responsável por selecionar, adaptar, verificar e sustentar tecnicamente o que será incorporado ao sistema.

Nesse cenário, a noção de competência profissional também se amplia. Saber programar deixa de significar apenas saber implementar manualmente cada solução e passa a envolver, cada vez mais, a capacidade de orientar ferramentas, formular boas instruções, identificar limitações, decidir quando aceitar ou rejeitar sugestões e responder pelas consequências dessas decisões. A atividade profissional se aproxima, assim, de funções como curadoria técnica, coordenação, análise e validação. Trata-se menos de substituição do desenvolvedor e mais de transformação qualitativa de sua atuação.

Dessa forma, a literatura e as fontes contextuais convergem para a ideia de que a principal mudança trazida pela IA generativa não está apenas no ritmo de produção de código, mas na reorganização das responsabilidades do trabalho técnico. O desenvolvedor permanece essencial, porém em uma posição progressivamente mais estratégica, crítica e orientada à supervisão. Essa reconfiguração ajuda a compreender por que o debate

sobre IA em software não deve se limitar à automação de tarefas, mas precisa considerar também a transformação mais ampla da prática profissional.

2.8 Síntese da revisão

A revisão da literatura permite observar que a incorporação da IA generativa no desenvolvimento de software não pode ser reduzida a uma simples narrativa de ganho de velocidade. Embora existam evidências de que essas ferramentas acelerem tarefas específicas e alterem positivamente certas etapas do fluxo de trabalho, a literatura também mostra que seus efeitos são desiguais, dependentes de contexto e marcados por limitações importantes. Ao mesmo tempo, os estudos analisados indicam que o impacto mais profundo da IA talvez não esteja apenas na produtividade, mas na transformação do papel do desenvolvedor, cuja atuação passa a depender cada vez mais de supervisão crítica, validação técnica e capacidade de decisão.

O capítulo seguinte explicita como as fontes foram delimitadas e analisadas. Com esse procedimento, torna-se possível avançar para a análise e discussão dos achados, relacionando as contribuições da literatura à hipótese central deste trabalho: a de que a IA generativa amplia possibilidades de produtividade em diversas tarefas, mas não substitui o desenvolvedor, reconfigurando sua atuação em direção a funções mais estratégicas, analíticas e críticas.

3 Metodologia

3.1 Caracterização da pesquisa

Este trabalho adota uma revisão narrativa da literatura com apoio documental e análise qualitativa. Seu objetivo é interpretar como a IA generativa interfere na produtividade, nas limitações do desenvolvimento de software e nas responsabilidades do desenvolvedor. A investigação articula resultados de pesquisas publicadas com contribuições conceituais e documentos técnicos, considerando as diferenças de finalidade e de alcance entre essas fontes.

A unidade de análise é a publicação e sua contribuição para o problema de pesquisa. Não foram realizados experimentos próprios, entrevistas, questionários ou coleta de dados com desenvolvedores. Quando esses procedimentos aparecem na revisão, pertencem aos estudos consultados, e seus resultados são examinados como evidências secundárias. A pesquisa também não realiza uma avaliação prática comparativa das ferramentas descritas.

O caráter narrativo decorre da seleção temática de um conjunto já reunido durante a elaboração do trabalho, sem protocolo de busca sistemática previamente registrado. Por isso, a revisão não pretende representar toda a produção científica sobre IA generativa nem estimar um efeito médio da tecnologia. Sua contribuição está na comparação crítica de perspectivas e na explicitação das condições que limitam a generalização dos achados.

3.2 Constituição e delimitação das fontes

O núcleo analítico reúne as onze publicações identificadas no Quadro 1, cujos anos bibliográficos vão de 2023 a 2026. Esse intervalo descreve o conjunto efetivamente utilizado; não corresponde a um filtro temporal de uma busca sistemática. As cinco fontes acadêmicas compreendem quatro pesquisas empíricas sobre produtividade e uma contribuição conceitual sobre desenvolvimento orientado por especificação. As outras seis publicações são técnicas, críticas ou jornalísticas.

Os registros bibliográficos permitem identificar o arXiv como meio de acesso às versões dos cinco trabalhos acadêmicos. Para o material técnico e contextual, os meios de acesso são o GitHub Blog, o site de Martin Fowler, o Technology Radar da Thoughtworks e a coluna do UOL Tilt indicada pelo orientador. Esses endereços documentam a obtenção das publicações, mas não demonstram que tenha sido realizada uma busca exaustiva nessas plataformas. As referências finais registram autoria, título, ano, endereço eletrônico e data de acesso disponível no cadastro do projeto.

A pertinência do conjunto foi examinada segundo três critérios: relação direta com pelo

menos um dos eixos do problema de pesquisa; identificação da autoria ou da organização responsável e possibilidade de consultar o conteúdo; e contribuição para compreender resultados, limites ou transformações do desenvolvimento com IA. A seleção contempla resultados favoráveis à aceleração de tarefas, resultados sem mudança estatisticamente significativa nas métricas observadas e evidências de aumento do tempo de execução. Essa diversidade permite confrontar a hipótese do trabalho com achados que a qualificam, em vez de reunir apenas argumentos favoráveis.

Na delimitação do núcleo, documentos sobre instalação e uso operacional de ferramentas, conteúdos sem relação direta com os eixos analisados e referências apenas cadastradas, mas não utilizadas na argumentação, não foram incorporados. Diferentes versões de um mesmo trabalho não foram contabilizadas como estudos independentes. Esses critérios explicitam a organização do conjunto final e não representam um procedimento retrospectivamente atribuído a uma triagem que não foi documentada.

As fontes acadêmicas recebem prioridade na discussão dos efeitos sobre produtividade. A contribuição conceitual sobre SDD não é tratada como experimento de eficácia, assim como a condição de *preprint* é distinguida da publicação em evento científico. Os textos técnicos permitem examinar propostas de organização do trabalho com agentes, enquanto a fonte jornalística contextualiza expectativas e percepções do mercado brasileiro. Essa composição é adequada aos diferentes aspectos do problema, desde que uma proposta de prática profissional não seja apresentada como demonstração empírica de ganho de desempenho.

Anúncios, documentação de produtos e registros oficiais de alterações compõem um conjunto documental complementar. Sua função é sustentar a contextualização histórica das ferramentas e a discussão de ambientes recentes em trabalhos futuros. Esses documentos não aumentam a contagem das onze fontes do núcleo analítico e não são utilizados como evidência independente de produtividade. A distinção é mantida na Seção 2.1 e nas citações ao longo do texto.

3.3 Procedimento de leitura e análise

A leitura comparativa foi organizada em cinco eixos: produtividade; limitações e retrabalho; transformação do papel do desenvolvedor; *Spec-Driven Development*; e *Harness Engineering*. Para cada estudo empírico, foram examinados o contexto, a tarefa ou atividade observada, o perfil dos participantes, o método, as métricas, o resultado principal e os limites reconhecidos. Para os textos conceituais e técnicos, foram considerados a proposta defendida, o tipo de apoio apresentado, as condições de aplicação e as ressalvas. A ausência de uma medida em uma fonte não foi preenchida por estimativas próprias.

A comparação separa redução de tempo em uma tarefa, atividade registrada em repositórios e percepção de produtividade. Não se somam percentuais nem se calcula uma

média entre resultados obtidos por métodos e métricas diferentes. Em vez disso, a análise discute o que cada procedimento permite observar e até onde seu resultado pode ser transportado para outros contextos de desenvolvimento.

Os conceitos de especificação e controle de agentes são relacionados a essa comparação como possibilidades de reorganização do processo. A análise distingue três níveis de afirmação: resultados relatados pelos estudos; propostas ou avaliações de autores técnicos; e interpretações construídas neste trabalho. Assim, a aproximação entre dificuldades empíricas e práticas como SDD não implica afirmar que essas práticas tenham sido testadas nos mesmos experimentos.

Na contextualização histórica, cada data é associada ao evento documentado: apresentação, acesso experimental ou disponibilização pública. Datas de atualização de páginas e de versões posteriores não são tomadas como lançamento inicial. Quando os documentos consultados não permitem confirmar esse marco, a ferramenta é descrita no texto sem receber uma data de estreia na figura. Os ambientes de julho de 2026 são discutidos como objetos de investigação futura, sem lhes atribuir os efeitos medidos em ferramentas anteriores.

3.4 Limitações do procedimento

Não há registro completo das expressões de busca, das plataformas inicialmente pesquisadas, das datas de todas as consultas nem do número de documentos localizados antes da seleção. Portanto, informa-se a quantidade final de onze fontes, mas não uma quantidade inicial desconhecida ou um fluxo de exclusões não registrado. Essa lacuna reduz a possibilidade de reprodução integral do levantamento e constitui uma limitação metodológica do trabalho.

A seleção narrativa também está sujeita a vieses de disponibilidade e de escolha. A combinação de experimentos, estudos organizacionais, propostas conceituais e documentos de mercado exige cuidado para não conferir o mesmo peso a evidências distintas. Além disso, a evolução das ferramentas limita a validade temporal das descrições. Essas restrições orientam uma síntese contextualizada, com conclusões proporcionais ao conjunto analisado, e são retomadas na Seção 4.8.

4 Análise e Discussão

4.1 Síntese dos principais achados

A leitura conjunta das onze fontes permite organizar a discussão em torno de uma tensão: a IA generativa amplia a capacidade de produzir soluções iniciais, mas a utilidade desse aumento depende do trabalho necessário para compreender, conferir e integrar essas soluções. A hipótese deste TCC encontra, assim, sustentação parcial e condicionada. Há evidência de aceleração em determinadas tarefas, porém o conjunto não sustenta um ganho universal de produtividade nem permite prever a substituição de uma profissão inteira.

Os estudos empíricos observam dimensões diferentes dessa transformação. Peng et al. [2023] investigam o tempo de conclusão de uma tarefa controlada; Becker et al. [2025], tarefas reais em projetos maduros; Stray et al. [2026], atividade em repositórios e percepção em uma organização; e Chen et al. [2026], fatores de produtividade percebidos por desenvolvedores. Compará-los exige considerar essas diferenças antes de classificar seus resultados como favoráveis ou contrários à IA.

As contribuições sobre especificação e controle de agentes acrescentam uma dimensão de processo a esse quadro. Piskala [2026] e Böckeler [2026] discutem como orientar e verificar a produção com IA. Neste trabalho, essas propostas são interpretadas como respostas possíveis às novas exigências de coordenação e validação, e não como comprovação de que qualquer equipe obterá melhores resultados ao adotá-las. A análise seguinte desenvolve essa distinção entre capacidade técnica, efeito observado e interpretação profissional.

4.2 O paradoxo da produtividade: velocidade versus produtividade real

O contraste entre Peng et al. [2023] e Becker et al. [2025] evidencia o problema de generalizar a partir de uma única situação. No primeiro experimento, a tarefa consistia em implementar um servidor HTTP em JavaScript, e o grupo com Copilot concluiu a atividade 55,8% mais rapidamente. No segundo, 16 desenvolvedores trabalharam em 246 tarefas de projetos de código aberto com os quais já tinham familiaridade; permitir o uso de IA aumentou o tempo de conclusão em 19%. As ferramentas, os participantes e as tarefas diferem, de modo que esses percentuais não medem um mesmo efeito em condições equivalentes.

A diferença não invalida nenhum dos resultados. Ela indica que acelerar a construção de uma solução delimitada e alterar um sistema existente são situações que impõem demandas distintas. Na interpretação deste trabalho, quanto mais a tarefa depende de

convenções e decisões acumuladas no projeto, menos adequado é tomar a velocidade de geração como aproximação suficiente do esforço total. Isso não estabelece uma regra de que projetos novos sempre ganham e projetos maduros sempre perdem produtividade; delimita uma condição que precisa ser examinada.

O estudo de Stray et al. [2026], com atividade de 39 profissionais da organização pública norueguesa NAV IT, acrescenta outra ressalva: os usuários que adotaram Copilot já apresentavam maior atividade antes da adoção, e não foi identificada mudança estatisticamente significativa nas métricas de atividade após sua introdução. Portanto, comparar apenas usuários e não usuários depois da adoção poderia atribuir à ferramenta uma diferença anterior a ela. Por outro lado, a ausência de significância nessas métricas não demonstra ausência de benefício em todas as dimensões do trabalho.

Essa distinção aproxima o estudo longitudinal da discussão de Chen et al. [2026], baseada em questionários e entrevistas na BNY Mellon, na qual produtividade envolve aspectos que registros de código não capturam integralmente. Redução de esforço percebido, compreensão e domínio do trabalho podem ter valor mesmo sem produzir mais alterações no repositório. O paradoxo se esclarece quando se reconhece que velocidade, volume de atividade e experiência de trabalho não são medidas intercambiáveis. A avaliação precisa explicitar qual delas está observando e quais ficam fora de seu alcance.

4.3 Retrabalho, validação e qualidade do software

A necessidade de validação altera a interpretação econômica do tempo economizado. Uma sugestão rápida pode ser útil como ponto de partida, mas também pode exigir leitura, correção e adaptação. O aumento do tempo observado por Becker et al. [2025] mostra que a disponibilidade da ferramenta não basta para assegurar economia líquida no contexto estudado. Não se deve, entretanto, converter esse resultado em uma taxa geral de defeitos: o desfecho principal do experimento é tempo de conclusão, não uma medida universal da qualidade do software produzido com IA.

A partir dessa distinção, este trabalho considera o retrabalho como uma dimensão necessária da avaliação. O esforço de gerar uma solução deve ser examinado junto ao esforço de torná-la aceitável. Por exemplo, obter uma implementação que passa em testes restritos pode não esclarecer se ela preserva uma regra do domínio ou se permanece compreensível para manutenção. Esse exemplo expressa uma preocupação analítica; não corresponde a um experimento conduzido neste TCC.

Böckeler [2026] oferece uma proposta técnica para organizar essa verificação por meio de controles e ciclos de retorno. A relação com os estudos de produtividade é de complementaridade: o resultado empírico delimita um problema de avaliação, enquanto a proposta técnica descreve formas de enfrentar parte dele. Não há, no conjunto anali-

sado, uma comparação controlada que permita afirmar quanto essa abordagem reduziria o retrabalho encontrado nos outros estudos.

Assim, qualidade não deve aparecer apenas como uma etapa posterior à produtividade. Se a entrega exige revisão extensa para poder ser utilizada, esse trabalho integra o custo da solução. Essa leitura também impede o extremo oposto de descartar toda geração automática por apresentar limitações: a questão é identificar quando o apoio oferecido compensa o esforço adicional de avaliação, com critérios adequados ao risco e à finalidade do sistema.

4.4 A transformação do papel do desenvolvedor

As fontes sobre SDD descrevem uma mudança no artefato que orienta o desenvolvimento: objetivos, restrições e comportamentos esperados tornam-se referências explícitas para implementação e verificação [PISKALA, 2026; DELIMARSKY, 2025]. Isso permite interpretar a transformação profissional para além da quantidade de código digitado. Formular um problema de modo verificável passa a ser parte do trabalho necessário para aproveitar a geração automática.

Essa interpretação não implica que especificar, revisar ou decidir sobre arquitetura sejam responsabilidades novas da engenharia de software. O que muda nos fluxos discutidos é sua relação com uma implementação parcialmente delegada. Quando a ferramenta propõe alterações, o profissional precisa avaliar tanto a adequação da solução quanto a qualidade da orientação que a produziu. A responsabilidade pela aceitação permanece distinta da execução automática da tarefa.

A ampliação das capacidades das ferramentas, descrita na Subseção 2.2.1, também não demonstra, por si só, uma transformação uniforme da ocupação. Autocompletar código, editar arquivos e delegar uma tarefa podem coexistir em uma mesma rotina. Por isso, a passagem para funções de supervisão deve ser entendida como reorganização possível da distribuição do esforço, sem presumir que todo desenvolvedor já tenha deixado de implementar diretamente.

O conjunto sustenta a importância de manter conhecimento técnico e capacidade crítica, mas não permite concluir que nenhum posto de trabalho possa ser reduzido ou substituído. A hipótese de permanência do desenvolvedor é defendida aqui no sentido de que especificação, julgamento e responsabilidade continuam necessários nos processos analisados. Isso é diferente de oferecer uma previsão quantitativa sobre emprego, que exigiria outra base de evidências.

4.5 Experiência profissional e dependência da IA

Experiência não deve ser tratada como uma variável simples, capaz de determinar sozinha o benefício da ferramenta. Familiaridade com programação, conhecimento do projeto e prática no uso de assistentes são dimensões diferentes. A amostra experiente de Becker et al. [2025] mostra que conhecer profundamente uma base de código não assegurou ganho de tempo com IA naquele experimento. Esse resultado não autoriza concluir que iniciantes sempre ganham mais ou que especialistas devam evitar a tecnologia.

Chen et al. [2026] amplia a discussão ao incluir competências técnicas e apropriação do trabalho entre as dimensões de longo prazo consideradas pelos desenvolvedores. Essa contribuição desloca a avaliação de uma entrega isolada para a capacidade de compreender e sustentar o que foi produzido. Contudo, relatos e percepções não demonstram, isoladamente, que o uso de IA cause perda de habilidade ao longo da carreira.

Na interpretação deste TCC, dependência problemática ocorre quando a facilidade de obter uma resposta não é acompanhada de condições para avaliá-la. O ponto relevante não é exigir que cada linha seja escrita manualmente, mas conservar a capacidade de explicar decisões, reconhecer falhas e retomar o trabalho quando a ferramenta não atende à necessidade. Trata-se de uma implicação para o uso e a formação profissional, não de um efeito causal medido nesta revisão.

Esse raciocínio também evita associar confiança apenas ao tempo de experiência. Um profissional pode conhecer a linguagem e ainda desconhecer restrições específicas do sistema; outro pode compreender o domínio e precisar de apoio em uma tecnologia nova. Avaliar o uso da IA exige considerar a combinação entre essas competências, a tarefa proposta e os mecanismos disponíveis para conferir seu resultado.

4.6 *Spec-Driven Development* e *Harness Engineering* como resposta à maior autonomia da IA

SDD e *Harness Engineering* abordam problemas relacionados, mas não idênticos. A especificação explicita o comportamento desejado; os mecanismos de controle ajudam a observar se a execução está coerente com esse referencial. A articulação entre ambos permite interpretar a autonomia como algo que depende de orientação e retorno, e não apenas da capacidade de um agente continuar produzindo alterações [PISKALA, 2026; BÖCKELER, 2026].

A proposta prática de Delimarsky [2025] dá materialidade a essa organização ao estruturar o desenvolvimento em torno de especificações e etapas de verificação. Entretanto, Böckeler [2025] identifica diferentes níveis de adoção e questões em aberto sobre manutenção das especificações e uso em bases existentes. O Technology Radar também situa SDD

como técnica a ser avaliada, evitando tratá-la como prática universalmente consolidada [Thoughtworks, 2025].

O contraponto reproduzido por Fowler [2026], a partir de uma observação de Kent Beck, questiona a expectativa de definir toda a especificação antes de aprender com a implementação. A consequência para esta análise é que maior centralidade da especificação não deve significar imobilidade. Se o desenvolvimento revela uma restrição antes desconhecida, é necessário rever tanto a implementação quanto o referencial que a orienta.

Há também um limite na própria validação: verificar coerência com uma especificação não garante que a especificação represente corretamente a necessidade do usuário. Na interpretação deste trabalho, uma falha compartilhada entre instruções e critérios de aceitação pode permanecer invisível para verificações construídas apenas a partir desses mesmos critérios. Por isso, a revisão humana inclui questionar objetivos e pressupostos, além de conferir conformidade do código.

A contribuição conjunta dessas abordagens é oferecer uma forma de discutir o processo que envolve o agente. Seus possíveis benefícios precisam ser ponderados em relação ao custo de produzir e manter especificações, testes e regras. A revisão não permite quantificar esse equilíbrio, mas indica por que a organização do trabalho constitui uma variável relevante para futuras avaliações de produtividade com IA.

4.7 Implicações para formação e mercado de trabalho

No plano da formação, a interpretação dos achados favorece o desenvolvimento articulado de competências de implementação, leitura de código, especificação e avaliação. A geração assistida pode integrar o aprendizado, mas a obtenção de uma resposta funcional não basta para demonstrar compreensão. Diante das dimensões de longo prazo levantadas por Chen et al. [2026], torna-se pertinente considerar também a capacidade de explicar e modificar uma solução, e não apenas a rapidez de sua produção.

Essa leitura sugere que atividades formativas possam valorizar justificativa de decisões, identificação de limites e elaboração de critérios de aceitação. São implicações propostas neste trabalho, não resultados de uma intervenção educacional. A revisão não comparou turmas nem acompanhou aquisição de competências ao longo do tempo, de modo que não determina qual estratégia pedagógica seria mais eficaz.

Quanto ao mercado, Gomes e Cortiz [2026] reúne percepções sobre pressão por produtividade, composição de equipes e mudanças no perfil esperado. A fonte documenta um debate profissional relevante, mas não constitui uma série estatística que permita atribuir variações de emprego exclusivamente à IA. Suas observações devem ser lidas como contexto, mantendo separação entre expectativas de empresas e efeitos demonstrados pelos estudos acadêmicos.

Relacionar esse contexto à heterogeneidade dos resultados empíricos conduz a uma implicação organizacional: metas de entrega baseadas apenas na disponibilidade de uma ferramenta podem ignorar a carga de integração e revisão. A decisão sobre adoção e formação deve considerar o processo completo. A literatura selecionada sustenta essa cautela analítica, mas não fornece percentuais universais para redução de equipes, aumento de capacidade ou valorização salarial.

4.8 Limitações da literatura analisada

A primeira limitação é a heterogeneidade. Os estudos diferem em métodos, participantes, ferramentas, tarefas e indicadores. O tempo de uma atividade controlada, o tempo de alterações em projetos maduros, a atividade de repositórios e a percepção dos usuários respondem a perguntas relacionadas, mas diferentes. Por esse motivo, a análise não calcula uma média entre resultados nem ordena as ferramentas por produtividade.

A segunda limitação diz respeito à transferência entre contextos. Resultados obtidos em determinadas organizações ou em projetos de código aberto não representam automaticamente estudantes, pequenas empresas ou todos os setores econômicos. Da mesma forma, observações sobre ferramentas de um período anterior não descrevem necessariamente agentes e ambientes lançados posteriormente. A cronologia documenta mudanças de disponibilidade e capacidade declarada, não uma sequência comprovada de melhorias de desempenho.

A terceira limitação está na natureza das fontes. O núcleo combina pesquisas empíricas, contribuição conceitual, versões em *preprint* e avaliações técnicas ou jornalísticas. Essa composição amplia os aspectos discutidos, mas exige distinguir observação, proposta e opinião. Em particular, descrições de produtos por seus responsáveis não substituem avaliações independentes, e propostas de SDD e controle de agentes não demonstram, por si, efeitos causais sobre produtividade.

Por fim, a seleção narrativa e a ausência de um registro completo do levantamento inicial, explicitadas no Capítulo 3, limitam a abrangência e a reprodutibilidade da revisão. O conjunto permite defender uma conclusão contextualizada: a IA generativa pode acelerar tarefas, mas seu valor depende de como geração, compreensão, especificação e validação são articuladas. Permanecem abertas questões sobre efeitos duradouros na qualidade, na aprendizagem e no trabalho profissional, retomadas nas propostas de trabalhos futuros.

5 Conclusões

Este trabalho teve como objetivo analisar como a IA generativa está impactando o desenvolvimento de software, especialmente em relação à produtividade, às limitações técnicas, à reorganização do processo de desenvolvimento e à transformação do papel do desenvolvedor. A partir da revisão da literatura, observa-se que a IA generativa já não pode ser tratada como uma tendência distante ou apenas como uma ferramenta experimental. Ela passou a interferir diretamente no fluxo de trabalho de desenvolvedores, equipes e empresas, modificando a forma como tarefas são planejadas, executadas, revisadas e validadas.

Uma das principais conclusões deste estudo é que a IA generativa pode ampliar a produtividade no desenvolvimento de software, mas esse efeito não é uniforme nem automático. O estudo de Peng et al. [2023] mostra ganhos relevantes em uma tarefa controlada com o uso do GitHub Copilot, o que sustenta a ideia de que assistentes de código podem acelerar atividades específicas. No entanto, outros estudos indicam que produtividade não pode ser reduzida apenas à velocidade de geração de código, pois envolve também qualidade da solução, aderência ao contexto, segurança, manutenibilidade e esforço necessário para revisar e corrigir o que foi produzido [STRAY et al., 2026; CHEN et al., 2026].

Nesse sentido, a discussão sobre produtividade com IA exige uma visão mais ampla. A produtividade pode ser percebida pelo desenvolvedor como fluidez, rapidez inicial ou facilidade para iniciar tarefas, mas pode não aparecer da mesma forma em métricas objetivas. Além disso, em projetos maiores, maduros ou com maior complexidade técnica, o uso da IA pode gerar retrabalho, aumentar a necessidade de validação e exigir maior esforço de revisão, como indica o estudo sobre desenvolvedores experientes em projetos de código aberto [BECKER et al., 2025]. Portanto, a pergunta central não deve ser apenas se a IA aumenta ou reduz produtividade, mas em quais condições ela contribui de fato para o desenvolvimento de software.

Outra conclusão importante é que a IA generativa não elimina a necessidade do desenvolvedor, mas altera de forma significativa sua atuação. O papel do profissional tende a se deslocar da escrita manual contínua de código para atividades mais estratégicas, como especificação, revisão, validação, arquitetura, análise de segurança e tomada de decisão técnica. A chamada "barreira do código" foi parcialmente reduzida, pois ferramentas generativas permitem que mais pessoas produzam trechos funcionais de software. Entretanto, transformar código gerado em um sistema confiável, seguro e sustentável continua exigindo conhecimento técnico, experiência e capacidade crítica.

Com isso, o desenvolvedor deixa de ser visto apenas como executor de código e passa a atuar como alguém que orienta, supervisiona e valida sistemas produzidos com apoio

de IA. Nesse novo cenário, tornam-se mais importantes competências como comunicação, compreensão de contexto, visão de produto, entendimento de arquitetura e capacidade de revisar soluções de forma crítica. O profissional que consegue articular diferentes áreas do conhecimento e transformar problemas mal definidos em especificações claras tende a ganhar relevância, pois o uso eficiente da IA depende cada vez mais da qualidade da orientação humana.

O eixo de *Spec-Driven Development* contribui para reforçar essa interpretação. Ao colocar a especificação como elemento central do desenvolvimento orientado por IA, o SDD mostra que o desafio não está apenas em escrever *prompts* melhores, mas em estruturar o trabalho antes da implementação. Definir objetivos, escopo, restrições, critérios de aceitação e contexto técnico passa a ser parte essencial do processo [PISKALA, 2026; DELIMARSKY, 2025]. Assim, o SDD ajuda a explicar por que o papel do desenvolvedor se aproxima cada vez mais de atividades de definição, coordenação e validação, e não apenas de codificação direta.

De forma complementar, a noção de *Harness Engineering* amplia essa análise ao mostrar que não basta especificar o que a IA deve fazer. Também é necessário construir mecanismos de controle ao redor dos agentes de código, como testes, *linters*, análise estática, revisão humana, *feedback* contínuo e restrições arquiteturais [BÖCKELER, 2026]. Essa perspectiva reforça que o uso produtivo da IA depende de um ambiente técnico bem estruturado. Quanto mais autonomia se concede aos agentes, maior se torna a necessidade de mecanismos de verificação e supervisão.

Portanto, a principal contribuição deste trabalho está em mostrar que o impacto da IA generativa no desenvolvimento de software não deve ser analisado apenas pela ótica da automação. A IA acelera tarefas, mas também desloca responsabilidades. Ela reduz parte do esforço de implementação, mas aumenta a importância da validação. Ela facilita a geração de código, mas torna mais evidente a necessidade de conhecimento técnico para avaliar o que foi produzido. Assim, a transformação mais relevante não está na substituição completa do desenvolvedor, mas na redefinição de seu papel.

Conclui-se, portanto, que a IA generativa ganhou espaço no desenvolvimento de software, mas seus efeitos dependem das condições de uso e permanecem em discussão. O futuro da área tende a ser marcado por maior integração entre desenvolvedores e sistemas inteligentes, mas também por novas exigências de especificação, supervisão, segurança e responsabilidade técnica. A evolução das ferramentas não elimina a incerteza sobre seus efeitos profissionais e organizacionais. Justamente por isso, o desenvolvedor que souber combinar base técnica, senso crítico e capacidade de orientar essas ferramentas terá papel central na engenharia de software dos próximos anos.

5.1 Trabalhos futuros

Os ambientes apresentados em julho de 2026, após a versão inicial do trabalho, oferecem objetos concretos para novas investigações. O Cobalt foi anunciado em 21 de julho como um ambiente em que tarefas de programação são executadas em um computador na nuvem, com alterações de código, execução de testes e retorno do resultado para revisão [Cobalt Team, 2026]. Já o registro de julho do CodeMote apresenta o Flightdeck como um espaço que reúne terminais, conversas com agentes e seus estados de execução [CodeMote, 2026]. Nesse segundo caso, trata-se de um novo recurso de um ambiente existente, e não do lançamento de um modelo de IA ou necessariamente da estreia do produto inteiro.

Esses anúncios documentam possibilidades de execução e supervisão, mas não constituem avaliações independentes de produtividade. As ferramentas não foram objeto dos estudos empíricos que fundamentam esta revisão. Sua inclusão, portanto, indica oportunidades de pesquisa, sem transportar para elas os ganhos ou as limitações medidos em outros produtos e períodos.

Uma investigação futura poderá comparar tarefas semelhantes com e sem esses ambientes, considerando o tempo total até uma entrega aceita, o esforço de revisão e correção, os defeitos identificados, a adequação aos requisitos e os custos de uso. Para ambientes que permitem acompanhar vários agentes, será relevante observar também a carga de supervisão e a coordenação entre alterações simultâneas. A avaliação deverá explicitar tarefa, experiência dos participantes, versão das ferramentas e critérios de aceitação, para permitir a interpretação contextualizada dos resultados.

Também são pertinentes estudos que acompanhem a manutenção das soluções e o domínio técnico do desenvolvedor ao longo do tempo. O objetivo é investigar se a ampliação da execução automática se traduz em benefícios sustentáveis, considerando segurança, compreensão do código e capacidade de corrigir falhas. Essas propostas constituem desdobramentos da revisão; não descrevem experimentos realizados neste TCC.

A continuidade das mudanças também exige examinar a viabilidade do uso de agentes: custos de infraestrutura, assinaturas e consumo de recursos podem alterar decisões de adoção. As propostas de especificação e controle oferecem caminhos para organizar esse trabalho, enquanto as discussões de mercado apontam mudanças nas expectativas sobre o profissional [BÖCKELER, 2025; BÖCKELER, 2026; GOMES; CORTIZ, 2026]. Investigar essas relações permitirá avaliar se maior capacidade de execução vem acompanhada de qualidade sustentável e de condições adequadas de supervisão. O futuro permanece aberto, e seu estudo exige critérios que ultrapassem a velocidade de geração de código.

Referências Bibliográficas

Anthropic. *Introducing Claude 4*. 2025. Anthropic. Disponível em: <https://www.anthropic.com/news/claude-4>. Acesso em: 2026-09-08.

Augment Code. *Introduction*. s.d. Documentação oficial do Augment Code. Disponível em: <https://docs.augmentcode.com/introduction>. Acesso em: 2026-09-07.

BECKER, J. et al. Measuring the impact of early-2025 ai on experienced open-source developer productivity. *arXiv preprint arXiv:2507.09089*, 2025. Disponível em: <https://arxiv.org/abs/2507.09089>. Acesso em: 2026-04-06.

BÖCKELER, B. *Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl*. 2025. Martinfowler.com. Disponível em: <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>. Acesso em: 2026-04-19.

BÖCKELER, B. *Harness engineering for coding agent users*. 2026. Martinfowler.com. Disponível em: <https://martinfowler.com/articles/harness-engineering.html>. Acesso em: 2026-05-25.

CHEN, V. et al. Beyond the commit: Developer perspectives on productivity with ai coding assistants. *arXiv preprint arXiv:2602.03593*, 2026. Disponível em: <https://arxiv.org/abs/2602.03593>. Acesso em: 2026-04-06.

Cobalt Team. *Introducing Cobalt: an AI developer with its own computer*. 2026. Cobalt Blog. Disponível em: <https://cobaltcode.ai/blog/introducing-cobalt>. Acesso em: 2026-09-08.

CodeMote. *Introducing Flightdeck*. 2026. What's new, version 2.1.2. Disponível em: <https://www.codemote.dev/whats-new>. Acesso em: 2026-09-08.

Cursor. *New inline edits*. 2023. Cursor Changelog, version 0.2.39. Disponível em: <https://cursor.com/changelog/0-2-39>. Acesso em: 2026-09-06.

DELIMARSKY, D. *Spec-driven development with AI: Get started with a new open source toolkit*. 2025. GitHub Blog. Disponível em: <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>. Acesso em: 2026-04-19.

DOHMKE, T. *GitHub Copilot is generally available to all developers*. 2022. GitHub Blog. Disponível em: <https://github.blog/news-insights/product-news/github-copilot-is-generally-available-to-all-developers/>. Acesso em: 2026-09-08.

FOWLER, M. *Fragments: January 8*. 2026. Martinfowler.com. Disponível em: <https://martinfowler.com/fragments/2026-01-08.html>. Acesso em: 2026-04-19.

GOMES, H. S.; CORTIZ, D. *IA revive medo da demissão: como dev é afetado por tecnologia feita por ele*. 2026. UOL Tilt. Disponível em: <https://www.uol.com.br/tilt/noticias/redacao/2026/03/24/deu-tilt-2403.ghtm>. Acesso em: 2026-04-06.

KELLY, C. *Meet Augment Code: Developer AI for teams*. 2024. Augment Code Blog. Disponível em: <https://www.augmentcode.com/blog/meet-augment-code-developer-ai-for-teams>. Acesso em: 2026-09-08.

OpenAI. *Codex is now generally available*. 2025. OpenAI. Disponível em: <https://openai.com/index/codex-now-generally-available/>. Acesso em: 2026-09-08.

OpenCode. *Intro*. s.d. Documentação oficial do OpenCode. Disponível em: <https://opencode.ai/docs/>. Acesso em: 2026-09-06.

PENG, S. et al. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590*, 2023. Disponível em: <https://arxiv.org/abs/2302.06590>. Acesso em: 2026-04-06.

PISKALA, D. B. Spec-driven development: From code to contract in the age of ai coding assistants. *arXiv preprint arXiv:2602.00180*, 2026. Disponível em: <https://arxiv.org/abs/2602.00180>. Acesso em: 2026-04-19.

RAAD, D. *Opencode rewrite is done and ready for general use*. 2025. X, perfil @thdxr. Disponível em: <https://x.com/thdxr/status/1935801226362302730>. Acesso em: 2026-09-08.

STRAY, V. et al. Developer productivity with and without github copilot: A longitudinal mixed-methods case study. *Proceedings of the 59th Hawaii International Conference on System Sciences (HICSS-59)*, 2026. Disponível em: <https://arxiv.org/abs/2509.20353>. Acesso em: 2026-04-06.

SWAMINATHAN, N.; ILINCA, D. *Kiro is generally available: Build with your team in the IDE and terminal*. 2025. Kiro Blog. Disponível em: <https://kiro.dev/blog/general-availability/>. Acesso em: 2026-09-08.

The Antigravity Team. *Introducing Google Antigravity, a New Era in AI-Assisted Software Development*. 2025. Google Antigravity Blog. Disponível em: <https://www.antigravity.google/blog/introducing-google-antigravity>. Acesso em: 2026-09-08.

The TRAE Team. *TRAE 2025 Unwrapped*. 2025. TRAE Blog. Disponível em: <https://www.trae.ai/blog/trae_2025_yearly_review>. Acesso em: 2026-09-06.

Thoughtworks. *Spec-driven development*. 2025. Technology Radar. Disponível em: <https://www.thoughtworks.com/en-us/radar/techniques/spec-driven-development>. Acesso em: 2026-04-19.